

ALMACEN PAJARO AZUL

MANUAL TÉCNICO

SISTEMA DE TRACKING DE PAQUETES APPTRACKINGPAQUETESAPA

Ing. Alexander Varela
Desarrollo de Aplicaciones de Vanguardia
13 de junio de 2026



PAJARO AZUL

PASION POR LA MUSICA

Manual Técnico

Este documento consolida la documentación técnica necesaria para instalar, ejecutar, validar, mantener y defender el sistema **appTrackingPaquetesAPA**. Está orientado a personal de sistemas, docentes evaluadores y responsables técnicos del proyecto.

PROYECTO

appTrackingPaquetesAPA

TIPO DE SOLUCIÓN

Aplicación web cliente-servidor

BACKEND

Node.js, Express, TypeScript

FRONTEND

Angular standalone

BASE DE DATOS

MongoDB con Mongoose

INFRAESTRUCTURA LOCAL

Docker Compose, MongoDB y Redis

Índice

- | | |
|-------------------------------|----------------------------|
| 1. Propósito y alcance | 13. Seguridad y roles |
| 2. Resumen del sistema | 14. Tiempo real |
| 3. Arquitectura general | 15. Auditoría |
| 4. Tecnologías utilizadas | 16. Reportes |
| 5. Estructura del repositorio | 17. Evidencias y archivos |
| 6. Requisitos de instalación | 18. Pruebas |
| 7. Variables de entorno | 19. CI/CD |
| 8. Ejecución local | 20. Backups y restauración |
| 9. Backend | 21. Despliegue sugerido |
| 10. Frontend | 22. Solución de problemas |
| 11. Modelo de datos | 23. Anexos |
| 12. API REST | |

1. Propósito y Alcance

El sistema permite registrar paquetes internos, dar seguimiento por número de guía, administrar catálogos operativos, registrar incidencias, subir evidencias, consultar reportes y mantener auditoría de acciones relevantes.

El alcance técnico incluye backend API REST, frontend Angular, persistencia en MongoDB, caché y mensajería auxiliar con Redis, comunicación en tiempo real con Socket.IO, autenticación JWT, autorización por roles y pipeline de validación en GitHub Actions.

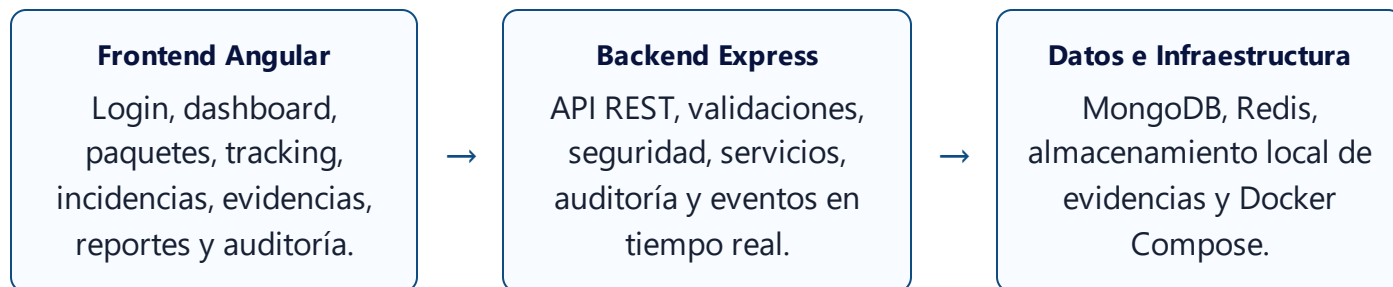
Alcance de este manual: instalación local, configuración, arquitectura, componentes, comandos, validaciones, mantenimiento, respaldo y criterios técnicos para defensa del Avance 3.

2. Resumen del Sistema

Área	Descripción
Gestión de paquetes	Alta, consulta, actualización y control de paquetes con número de guía.
Tracking	Historial de avances por paquete y por guía.
Incidencias	Registro de problemas operativos asociados a un paquete.
Evidencias	Carga y consulta de archivos JPG, PNG y PDF relacionados con el proceso.
Reportes	Salidas de información para paquetes por estado, incidencias y actividad.
Auditoría	Registro de acciones relevantes para trazabilidad administrativa.

3. Arquitectura General

La aplicación sigue una separación clara entre cliente, servidor y servicios de infraestructura. El frontend consume la API del backend. El backend aplica reglas de negocio y persiste información en MongoDB. Redis se utiliza como soporte para caché y tiempo real, con tolerancia a fallo para no bloquear la operación principal.



Capas del backend

Capa	Carpeta	Responsabilidad
Configuración	backend/src/config	Variables de entorno, conexión a MongoDB y Redis.
Rutas	backend/src/rutas	Definición de endpoints REST.
Controladores	backend/src/controladores	Reciben solicitudes HTTP y delegan al servicio correspondiente.
Servicios	backend/src/servicios	Reglas de negocio, persistencia y operaciones principales.
Modelos	backend/src/modelos	Esquemas Mongoose y definición de índices.
Middlewares	backend/src/middlewares	Autenticación, roles, validación, subida de archivos y errores.
Validadores	backend/src/validadores	Esquemas Zod para validar entradas.
Tiempo real	backend/src/realtime	Socket.IO, rooms, eventos y Redis adapter con fallback.

4. Tecnologías Utilizadas

Componente	Tecnología	Uso
Backend	Node.js, Express, TypeScript	Servidor API y lógica de negocio tipada.
Base de datos	MongoDB, Mongoose	Persistencia documental, esquemas e índices.
Caché / soporte	Redis	Caché operativo y soporte para tiempo real.
Autenticación	JWT, bcryptjs	Sesiones seguras y contraseñas cifradas.
Validación	Zod	Validación de entradas antes de procesarlas.
Archivos	Multer	Carga de evidencias.
Tiempo real	Socket.IO	Notificación de cambios relevantes.
Frontend	Angular, RxJS, PrimeNG, SCSS	Interfaz web, rutas, formularios y servicios HTTP.
Pruebas	Jest, Supertest, Playwright	Pruebas backend y base E2E frontend.
CI/CD	GitHub Actions	Build y pruebas automáticas en la rama principal.

5. Estructura del Repositorio

```
appTrackingPaquetesAPA/  
  backend/           API REST, Socket.IO, modelos, servicios y pruebas  
  frontend/         Aplicación Angular y pruebas E2E  
  documentacion/     Manuales, diagramas, estado del proyecto y guías  
  scripts/backup/    Scripts de respaldo y restauración  
  docker-compose.yml MongoDB y Redis para entorno local  
  .github/workflows/ci.yml Pipeline de validación automática
```

La estructura separa responsabilidades y facilita la defensa técnica: backend y frontend pueden compilarse, probarse y desplegarse como componentes independientes.

6. Requisitos de Instalación

Requisito	Versión / condición recomendada	Uso
Node.js	22.x recomendado para CI	Instalación de dependencias y ejecución de backend/frontend.
npm	Incluido con Node.js	Gestión de paquetes y scripts.
Docker Desktop	Activo antes de ejecutar la app	Levantar MongoDB y Redis.
Git	Configurado con acceso al repositorio	Clonar, versionar y publicar cambios.
Navegador	Chrome, Edge o equivalente	Uso de la aplicación web.

7. Variables de Entorno

El backend utiliza `backend/.env`. El archivo base es `backend/.env.example`.

```
NODE_ENV=development
PUERTO=4300
PORT=4300
MONGO_URI=mongodb://127.0.0.1:27017/appTrackingPaquetesAPA
MONGODB_URI=mongodb://127.0.0.1:27017/appTrackingPaquetesAPA
REDIS_URL=redis://127.0.0.1:6379
CORS_ORIGINS=http://localhost:3180
FRONTEND_URL=http://localhost:3180
UPLOAD_DIR=uploads/evidencias
JWT_SECRET=cambiar_esta_clave_en_produccion
JWT_EXPIRES_IN=1d
```

Importante: en producción, `JWT_SECRET` debe ser una clave segura y no debe versionarse. Los secretos deben administrarse con variables de entorno del servidor o del proveedor cloud.

8. Ejecución Local

8.1 Levantar infraestructura

```
docker compose up -d
```

8.2 Backend

```
cd backend
npm install
copy .env.example .env
npm run seed:demo
npm run dev
```

8.3 Frontend

```
cd frontend
npm install
npm start
```

8.4 URLs locales

Servicio	URL	Uso
Frontend	http://localhost:3180/login	Acceso a la aplicación.
Backend salud	http://localhost:4300/api/salud	Validar que la API responde.
MongoDB	localhost:27017	Base de datos local.
Redis	localhost:6379	Caché y soporte de eventos.

9. Backend

El backend expone una API REST protegida por JWT. Su entrada principal es `src/servidor.ts`, que conecta MongoDB, Redis y configura Socket.IO sobre el servidor HTTP.

Scripts principales

Comando	Descripción
<code>npm run dev</code>	Ejecuta el backend en modo desarrollo con recarga.
<code>npm run build</code>	Compila TypeScript usando <code>tsconfig.build.json</code> .
<code>npm start</code>	Ejecuta el backend compilado desde <code>dist/</code> .
<code>npm run seed:demo</code>	Carga datos demo idempotentes.
<code>npm test</code>	Ejecuta pruebas con Jest.

Servicios principales

Servicio	Responsabilidad
<code>auth.servicio.ts</code>	Login, validación de contraseña y emisión de JWT.
<code>paquete.servicio.ts</code>	Registro, consulta y actualización de paquetes.
<code>tracking.servicio.ts</code>	Historial de avances por paquete y guía.
<code>incidencia.servicio.ts</code>	Gestión de incidencias operativas.
<code>evidencia.servicio.ts</code>	Carga y consulta de evidencias.
<code>dashboard.servicio.ts</code>	Resumen operativo del tablero.
<code>auditLog.servicio.ts</code>	Registro de acciones auditables.
<code>reporte.servicio.ts</code>	Consultas agregadas para reportes.
<code>accesoPaquete.servicio.ts</code>	Control por rol y propiedad del paquete.

10. Frontend

El frontend está construido con Angular standalone. Utiliza rutas protegidas, servicios HTTP, interceptor de token, componentes por módulo funcional y conexión al canal de tiempo real.

Rutas visibles

Ruta	Pantalla
/login	Inicio de sesión.
/dashboard	Resumen operativo.
/paquetes	Listado y búsqueda de paquetes.
/paquetes/nuevo	Registro de paquete.
/paquetes/:id	Detalle del paquete.
/tracking:numeroGuia	Seguimiento por guía.
/incidencias	Gestión de incidencias.
/evidencias	Carga y consulta de evidencias.
/usuarios	Administración de usuarios.
/lugares	Administración de lugares.
/estados	Administración de estados.
/reportes	Salidas de información.
/auditoria	Registro de auditoría.

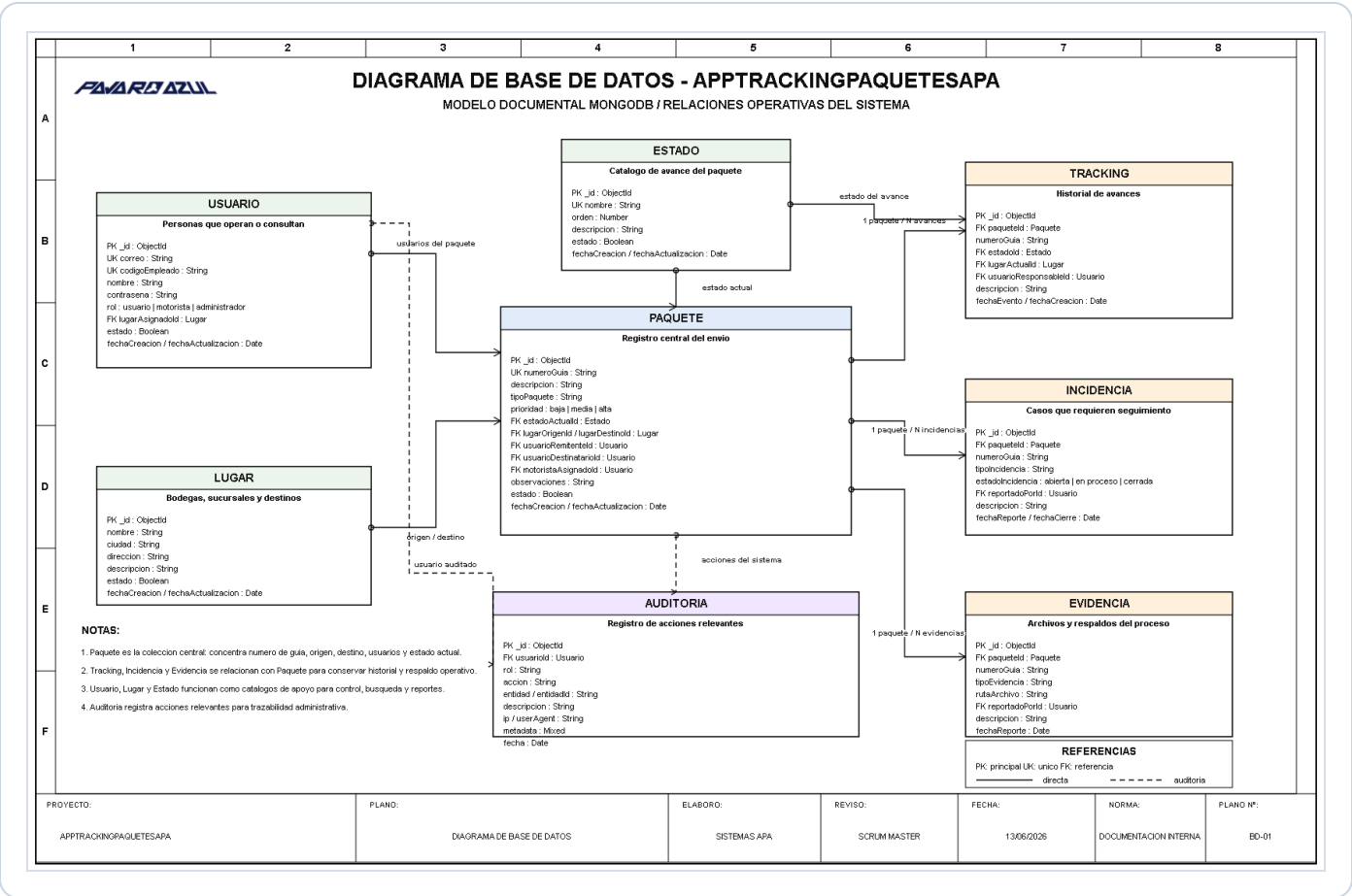
Scripts principales

```
npm start      # ng serve --port 3180 --proxy-config proxy.conf.json
npm run build  # build de producción
npm run e2e    # pruebas Playwright
```

11. Modelo de Datos

MongoDB almacena documentos por colección. Mongoose define esquemas, validaciones básicas e índices. La colección central es `Paquete`, relacionada con usuarios, lugares, estados, tracking, incidencias y evidencias.

Colección	Uso	Relaciones principales
usuarios	Empleados, motoristas y administradores.	Lugar asignado, paquetes, tracking, incidencias, evidencias y auditoría.
lugares	Bodegas, sucursales y destinos.	Origen, destino y ubicación actual.
estados	Flujo del paquete.	Estado actual y estado de tracking.
paquetes	Registro central del envío.	Usuarios, lugares, estado, tracking, incidencias y evidencias.
trackings	Historial de avances.	Paquete, estado, lugar y usuario responsable.
incidencias	Problemas o eventos de seguimiento.	Paquete y usuario que reporta.
evidencias	Archivos de respaldo.	Paquete y usuario que reporta.
auditlogs	Trazabilidad administrativa.	Usuario, entidad, acción y metadatos.



12. API REST

La API utiliza la base `/api`. Los endpoints protegidos requieren encabezado `Authorization: Bearer <token>`.

Módulo	Endpoints principales	Roles
Salud	GET <code>/salud</code>	Público
Auth	POST <code>/auth/login</code>	Público
Dashboard	GET <code>/dashboard/resumen</code>	Administrador
Usuarios	GET/POST/PUT/DELETE <code>/usuarios</code>	Administrador
Lugares	GET/POST/PUT/DELETE <code>/lugares</code>	Autenticado / Administrador
Estados	GET/POST/PUT/DELETE <code>/estados</code>	Autenticado / Administrador
Paquetes	GET <code>/paquetes</code> , POST <code>/paquetes</code> , PUT <code>/paquetes/:id</code>	Según rol y propiedad
Tracking	GET <code>/tracking/guia/:numeroGuia</code> , POST <code>/tracking</code>	Autenticado / Motorista / Administrador
Incidencias	GET <code>/incidencias</code> , POST <code>/incidencias</code>	Administrador / Usuario / Motorista
Evidencias	GET <code>/evidencias</code> , POST <code>/evidencias/upload</code>	Autenticado
Reportes	GET <code>/reportes/*</code>	Administrador
Auditoría	GET <code>/auditoria</code>	Administrador

13. Seguridad y Roles

Mecanismo	Implementación	Objetivo
JWT	<code>auth.middleware.ts</code> e interceptor frontend	Proteger rutas internas y endpoints privados.
Bcrypt	<code>auth.servicio.ts</code>	Comparar contraseñas cifradas.
Roles	<code>rolMiddleware.ts</code>	Limitar acciones administrativas u operativas.
Propiedad de recurso	<code>accesoPaquete.servicio.ts</code>	Evitar que usuarios consulten paquetes no relacionados.
Helmet y CORS	<code>app.ts</code>	Endurecer cabeceras HTTP y controlar orígenes.
Rate limit	<code>rateLimitMiddleware.ts</code>	Reducir abuso en endpoints sensibles.
Zod	<code>validadores/</code>	Validar entradas y devolver errores consistentes.

El diseño protege tanto rutas frontend como endpoints backend. La seguridad real se aplica en el servidor; el frontend solo mejora la experiencia de navegación.

14. Tiempo Real

Socket.IO permite notificar cambios relevantes a pantallas abiertas. El backend autentica la conexión mediante JWT y organiza clientes por salas como usuario, rol, paquete y guía.

Evento	Uso esperado
<code>dashboard:updated</code>	Actualizar métricas del tablero.
<code>package:created</code>	Notificar creación de paquete.
<code>package:updated</code>	Notificar actualización de paquete.
<code>tracking:created</code>	Actualizar historial de seguimiento.
<code>incidence:created</code>	Notificar incidencia nueva.
<code>evidence:created</code>	Notificar evidencia nueva.

15. Auditoría

La auditoría registra acciones sensibles con usuario, rol, acción, entidad, descripción, IP, user agent, metadatos y fecha. Esto permite rastrear cambios y apoyar la defensa del control administrativo.

Eventos auditable: login exitoso/fallido, creación y actualización de entidades, cambios en paquetes, tracking, incidencias, evidencias y operaciones administrativas.

16. Reportes

Reporte	Objetivo
Paquetes por estado	Conocer distribución operativa del flujo de paquetes.
Incidencias	Revisar casos abiertos, en proceso o cerrados.
Actividad reciente	Evaluar movimientos y acciones recientes del sistema.

17. Evidencias y Archivos

Las evidencias se cargan con Multer y se almacenan bajo la ruta configurada en `UPLOAD_DIR`. El modelo guarda tipo, descripción, ruta del archivo, paquete, guía, usuario que reporta y fecha.

Control	Detalle
Formatos permitidos	JPG, JPEG, PNG y PDF.
Asociación	La evidencia queda relacionada con un paquete y un número de guía.
Acceso	La descarga valida permisos sobre el paquete relacionado.
Producción	Se recomienda migrar a almacenamiento externo como S3, Azure Blob o equivalente.

18. Pruebas y Validación

Backend

```
cd backend
npm run build
npm test -- --runInBand
```

Las pruebas cubren salud de API, autenticación, paquetes y acceso por propiedad de recurso.

Frontend

```
cd frontend
npm run build
npm run e2e
```

El build valida que Angular compile correctamente. Las pruebas E2E requieren backend, frontend y datos demo activos.

Validación manual mínima

1. Levantar Docker, backend y frontend.
2. Entrar con `Sistemas / Sistemas*2026`.
3. Revisar dashboard, paquetes, tracking, incidencias, evidencias, reportes y auditoría.
4. Crear un paquete de prueba y verificar que aparezca en listado y tracking.
5. Registrar una incidencia y una evidencia.

19. CI/CD

El repositorio cuenta con workflow de GitHub Actions en `.github/workflows/ci.yml`. Se ejecuta en push y pull request hacia `main`.

Paso	Acción
Checkout	Descarga el repositorio.
Setup Node.js	Configura Node 22 y caché de npm.
Backend install	Ejecuta <code>npm ci</code> en backend.
Backend build	Ejecuta <code>npm run build</code> .
Backend test	Ejecuta <code>npm test -- --runInBand</code> .
Frontend install	Ejecuta <code>npm ci</code> en frontend.
Frontend build	Ejecuta <code>npm run build</code> .

20. Backups y Restauración

El proyecto incluye scripts en `scripts/backup/` para respaldar y restaurar MongoDB. La estrategia recomendada es respaldar antes de cambios importantes, despliegues o pruebas destructivas.

```
scripts\backup\mongo-backup.cmd
scripts\backup\mongo-restore.cmd
```

Antes de una defensa o presentación, se recomienda probar restauración en una base temporal para confirmar que el respaldo es utilizable.

21. Despliegue Sugerido

Elemento	Recomendación para producción
Frontend	Compilar con <code>npm run build</code> y servir desde Nginx, Apache o hosting estático.
Backend	Ejecutar <code>npm run build</code> y <code>npm start</code> bajo PM2, systemd o contenedor.
Dominio y TLS	Usar HTTPS obligatorio con certificado válido.
Proxy reverso	Nginx o equivalente para enrutar API, frontend y Socket.IO.
Secretos	Variables seguras fuera del repositorio.
Archivos	Almacenamiento externo para evidencias.
Observabilidad	Logs centralizados, métricas y alertas.

22. Solución de Problemas

Problema	Causa probable	Acción sugerida
Backend no inicia	MongoDB no disponible o variables incompletas.	Revisar Docker, <code>.env</code> y <code>/api/salud</code> .
Frontend no carga datos	Backend apagado o proxy no configurado.	Confirmar backend en puerto 4300 y reiniciar <code>npm start</code> .
Login falla	Credenciales incorrectas o seed no ejecutado.	Ejecutar <code>npm run seed:demo</code> y probar usuario Sistemas.
Evidencias no suben	Archivo no permitido o carpeta sin permisos.	Usar JPG, PNG o PDF y revisar <code>UPLOAD_DIR</code> .
Tiempo real no actualiza	Socket.IO o Redis no disponible.	Revisar consola, proxy <code>/socket.io</code> y estado de Redis.

23. Anexos

Anexo A. Comandos de referencia

```
docker compose up -d
docker compose ps

cd backend
npm run build
npm test -- --runInBand
npm run seed:demo
npm run dev

cd frontend
npm run build
npm start
npm run e2e
```

Anexo B. Credenciales demo

Campo	Valor
Usuario visible	Sistemas
Correo API	sistemas@pajaroazul.local
Contraseña	Sistemas*2026
Rol	administrador

Anexo C. Criterios de listo técnico

- Docker levanta MongoDB y Redis sin conflicto de puertos.
- Backend compila y responde en `/api/salud`.
- Frontend compila y permite login.
- Seed demo crea datos iniciales sin duplicar registros.
- CRUD principal, tracking, incidencias, evidencias, reportes y auditoría funcionan.
- GitHub Actions valida build y pruebas en `main`.

